

LPU 3D Campus Navigator

System Instructions / Developer Rules

1. Core Principle

- Build the smallest reliable feature satisfying the product requirement.
- Avoid unnecessary libraries, abstractions and services.
- Prefer simple, maintainable solutions.

2. Required Tech Stack

- Frontend: Next.js, TypeScript, React, Tailwind CSS
- 3D: Three.js, React Three Fiber, @react-three/drei
- Backend: Next.js server-side functionality/API routes where appropriate
- Database: Supabase + PostgreSQL
- Authentication: Supabase Auth
- AI: server-side AI API integration
- Validation: Zod
- Package manager: npm

3. TypeScript Rules

- Use TypeScript throughout.
- Avoid any; prefer explicit types.
- Validate external data.
- Do not use @ts-ignore unless unavoidable.
- Shared types live in /types.

4. React & Next.js Rules

- Functional components and appropriate hooks.
- Server Components by default; use client components only for browser interaction.
- Separate UI, data access and business logic.
- Never expose secrets to client components.
- Do not use deprecated APIs.

5. Database Rules

- PostgreSQL/Supabase is the source of truth.
- Use migrations and foreign keys.
- Do not hard-code production data into UI components.
- Use indexes for frequently searched fields.
- Use Row Level Security where appropriate.
- Never expose service-role credentials to the client.

6. 3D Map Rules

- Optimize GLB/GLTF assets.
- Lazy-load large assets.
- Avoid unnecessary rendering.
- Every interactive building has a stable ID mapped to a database location ID.
- Do not duplicate location information inside the model.

7. Routing Rules

- Use a graph of nodes and walkable edges.
- MVP: Dijkstra's algorithm.
- Future optimization: A*.
- Do not use straight-line distance as the walking route.

8. AI Chatbot Rules

- Use verified application data.
- Never fabricate building names, vendors, hours, facilities or directions.
- If unavailable: 'I don't have verified information about that yet.'
- AI API calls happen server-side.
- Protect and rate-limit API endpoints where appropriate.

9. UI Rules

- Modern, minimal, fast and mobile-first.
- 3D map is the primary interface.
- Avoid excessive gradients, animation, popups and decorative effects.
- Animations communicate selection, loading, route creation and transitions.

10. Accessibility

- Keyboard navigation where applicable.
- Visible focus states.
- Sufficient contrast.
- Descriptive labels.
- Do not rely only on color.
- Provide a 2D/list alternative for important map information.

11. Performance

- Fast initial load.
- Compress images and 3D assets.
- Cache relatively static campus data.
- Debounce search.
- Avoid rerendering the whole 3D scene for UI-only changes.

12. Security

- Never commit API keys, database passwords, service-role keys or auth secrets.
- Validate all user input.
- Apply server-side authorization to protected operations.

13. AI Vibe-Coding Rules

- Check existing folders, components and types before adding code.
- Reuse utilities.
- Do not install a library unless necessary.
- Do not create duplicate functionality.
- Explain large architectural changes first.
- Preserve working functionality.
- Do not rewrite unrelated files.

14. Definition of Done

- Desktop works.
- Mobile works where applicable.
- TypeScript passes.
- No new console errors.
- Loading/error/empty states exist.
- Database errors are handled.
- Existing functionality remains intact.
- Feature matches the PRD.

Recommended folder structure

```
/app  
/components  
/lib  
/types  
/data  
/public  
/supabase  
/tests  
/docs  
.env.example  
README.md
```